Search and Retrieval

Finding Things for Agents and Humans

Matt Redmond

25 Feb 2026

Today's Journey

1. **What Is Search?**
   - Documents, queries, and the core abstractions
2. **Lexical Search**
   - Preprocessing, tokenization, indexing, TF-IDF, BM25
3. **Semantic Search**
   - Embeddings, vector indices, approximate nearest neighbors
4. **Hybrid Search**
   - Combining lexical and semantic results: RRF, reranking
5. **Agentic Search**
   - LLM agents with search tools, query rewriting
6. **Non-Query Signals**
   - Recency, authority, PageRank, personalization

Outline

1. What Is Search?
2. Lexical Search
3. Semantic Search
4. Hybrid Search
5. Agentic Search
6. Non-Query Signals
7. Discussion
8. Conclusion

The Goal of Search

"To organize the world's information and make it universally accessible and useful." *Google's 2001 mission statement*

Documents

Information is assumed to be present in **documents**—the objects we are interested in finding.

Queries

Users look for documents with **queries**—expressions of intent that the search system must interpret.

A search system processes a **query** and returns an ordered (possibly empty) collection of **documents**.

Documents: The Core Abstraction

Most search systems make an assumption: the objects we are interested in are represented by a **Document** type.

- Documents are typically represented as groups of named key–value **fields**
- Documents have **types**, and we store similar-typed documents in **collections**

Example Document Schema

```
@dataclass
class Document:
    id: UUID
    author_id: UUID
    name: str                   # indexed for lexical search
    description: str | None     # indexed for lexical search
    created_at: datetime        # useful for filtering
    endorsed: bool              # useful for filtering
```

Queries: From Unstructured to Structured

Typically, to indicate their intent, a user issues an "unstructured query"—a string literal propagated into the search system from the top. But queries don't *have* to be single strings.

Simple (unstructured) query

```
"find me all blubs"
```

Structured query

```
{
  "query": {
    "match": {
      "object_type": "blub"
    }
  }
}
```

The order of returned documents is determined by a **ranking function**—the heart of any search system.

A Few Kinds of Structured Queries

Basic Match

Searches for a term within a specific field.

```
{"query": {"match":
  {"field": "text"}}}
```

Uses: Lexical Index

Exact Phrase

Searches for an exact phrase, maintaining term order.

```
{"query":
  {"match_phrase":
    {"field": "some text"}}}
```

Uses: Lexical Index

Range Query

Filters on numeric or date fields.

```
{"query": {"range":
  {"num_field":
    {"gte":10,"lte":50}}}}
```

Uses: Numeric / BTree Index

Outline

1. What Is Search?
2. Lexical Search
3. Semantic Search
4. Hybrid Search
5. Agentic Search
6. Non-Query Signals
7. Discussion
8. Conclusion

What Is Lexical Search?

Lexical Search uses the **text content** of queries to identify documents with similar **text**.

Two phases:

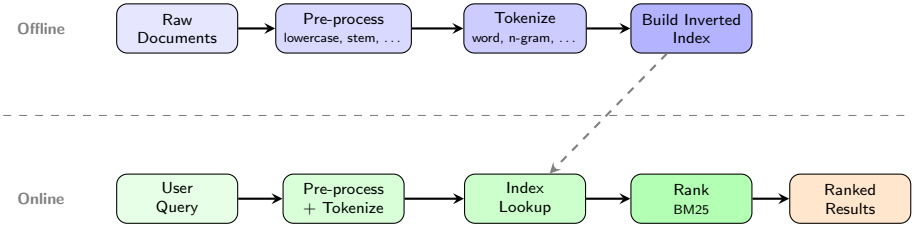
Setup (offline)

1. Document pre-processing
2. Document tokenization
3. Indexing

Runtime (online)

1. Query pre-processing + tokenization
2. Ranking

Lexical Search: The Full Pipeline



The next slides walk through each box in detail. Keep this picture in mind—every step at query time mirrors an indexing step.

Document Pre-Processing

01 — Lowercasing

- Transform fields to all lowercase
- Skip if capitalization disambiguates documents

02 — Punctuation Removal

- Replace punctuation with whitespace
- Skip if punctuation is meaningful (e.g. code!)

03 — Stemming

- Convert words to their root form (e.g. "running" → "run")

04 — Stop Word Removal

- Remove filler words: "a", "the", "of", ...

Important

All of these transformations are **optional**. The right choices depend on your corpus. The devil is in the details—poor preprocessing is a top cause of missing search results.

Document Tokenization

01 — Word Level

- Split on whitespace to produce "word" tokens

02 — Character Level

- Split at character boundaries

03 — Subword Level

- Split inside words:
  "unhappiness" $\rightarrow$ ["un", "happiness"]
- Similar to LLM tokenization, but not the same tokens!

04 — N-gram Level

- Break strings into sequences of n-grams
  e.g. $n = 4$: "unhappiness" $\rightarrow$ ["unha", "nhap", "happ", ..., "ness"]

Worked Example: Pre-Processing in Action

Document:

```
{"id": "ff1f...", "name": "Barry's Forecast of Revenue",
 "description": "Sometimes you really want to know how much
  money you have. Sometimes you need to know how much money
  you WILL have."}
```

"name" field config:

- Lowercase, remove punctuation
- Tokenize: 5-gram (character)

Preprocessed: `barrys forecast of revenue`
Tokens: `["barry","arrys","rrys ","rys f", ..., "venue"]`

"description" field config:

- Lowercase, remove punctuation, remove stopwords
- Tokenize: word boundaries

Preprocessed: `sometimes really want know much money sometimes need know much money`
Tokens: `["sometimes","really","want", "know","much","money",...]`

From Tokens to Sparse Vectors

After processing and tokenizing each field of each document, we collect all tokens seen for that field into a **universal vocabulary**.

Each document field becomes a **sparse vector** where each vocabulary element is a dimension, and the value is the token's count in that field.

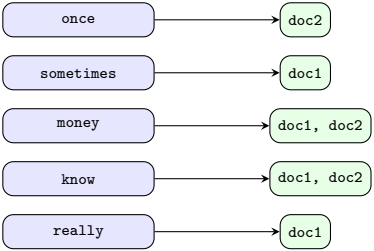
Example

Vocabulary: ["once", "sometimes", "money", "know", "since", "really", "need", "much", "want", "forever", "cold", ...]

doc1 description: [0, 2, 2, 2, 0, 1, 1, 2, 1, 0, 0, ...]
doc2 description: [1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, ...]

Indexing: The Inverted Index

The sparse vectors from the previous slide are *row-oriented*: one vector per document. An **inverted index** (also called a **posting list**) is the *column-oriented* view of the same data—a mapping from each token to the set of documents that contain it.



Why an inverted index?

- Given a query token, we can instantly look up which documents contain it
- No need to scan every document at query time
- This is the data structure behind most text search engines (Lucene, Elasticsearch, etc.)

Querying w/ Unstructured Queries

At query time, we take the user's query and apply the **same pre-processing steps** for each field we are searching over.

Example

Raw query: "barry cold money forecast"
Query for "name" field (5-gram): ["barry ", "arry ", "rry c", . . . , "ecast"]
Query for "desc" field (word): ["barry", "cold", "money", "forecast"]

Key Question

What do you do with query terms that *aren't* in the index? They contribute zero to the ranking score—the document can't match on a term it never contained.

Ranking: TF-IDF

TF-IDF is a simple ranking function: the product of **term frequency** and **inverse document frequency**.

Definitions

- $\mathrm{TF}(t, d)$ = number of times term t appears in document d
- $\mathrm{DF}(t)$ = number of documents containing t
- $\mathrm{IDF}(t) = \log\left(\frac{N}{1+\mathrm{DF}(t)}\right)$
  where N = total documents

Scoring

For each query term t and document d:

$$\mathrm{TFIDF}(t, d) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t)$$

Sum over all query terms to get the document's total score.

Intuition

Terms that appear frequently *in a document* but rarely *across the corpus* are the most useful for distinguishing relevant documents. IDF can be **pre-computed**.

TF-IDF: Worked Example

Query tokens: ["barry", "cold", "money", "forecast"] ($N = 2$ documents)

doc1 scores:

- barry: $\mathrm{TF} = 0 \Rightarrow 0$
- cold: $\mathrm{TF} = 0 \Rightarrow 0$
- money: $\mathrm{TF} = 2$, $\mathrm{IDF} = \log(2/3) = -0.176 \Rightarrow -0.352$
- forecast: $\mathrm{TF} = 0 \Rightarrow 0$

Total: -0.352

doc2 scores:

- barry: $\mathrm{TF} = 0 \Rightarrow 0$
- cold: $\mathrm{TF} = 1$, $\mathrm{IDF} = \log(2/2) = 0 \Rightarrow 0$
- money: $\mathrm{TF} = 1$, $\mathrm{IDF} = -0.176 \Rightarrow -0.176$
- forecast: $\mathrm{TF} = 0 \Rightarrow 0$

Total: $-0.176 \leftarrow$ **better match**

Problems with TF-IDF

1. **Term frequency saturation:** TF-IDF grows unboundedly with term frequency—a term appearing 100 times has disproportionate weight compared to 10 times, even though relevance plateaus in practice
2. **Document length bias:** Doesn't properly account for document length; penalizes longer documents that naturally contain more repetitions
3. **No tunable parameters:** Rigid formula with no way to adjust saturation or length normalization for different collections
4. **Poor handling of common terms:** IDF doesn't sufficiently dampen very common terms
5. **Linear term frequency scaling:** Doesn't match actual relevance patterns observed in practice

Ranking: BM25

BM25 is the modern successor to TF-IDF, addressing its key weaknesses.

$$\mathrm{BM25}(t, d) = \mathrm{IDF}(t) \times \frac{\mathrm{TF}(t, d) \cdot (k_1 + 1)}{\mathrm{TF}(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)}$$

Key improvements over TF-IDF:

- **Saturation curve:** Diminishing returns after ∼5–10 occurrences
- **Length normalization:** Adjustable penalty for long documents
- **Tunable parameters**

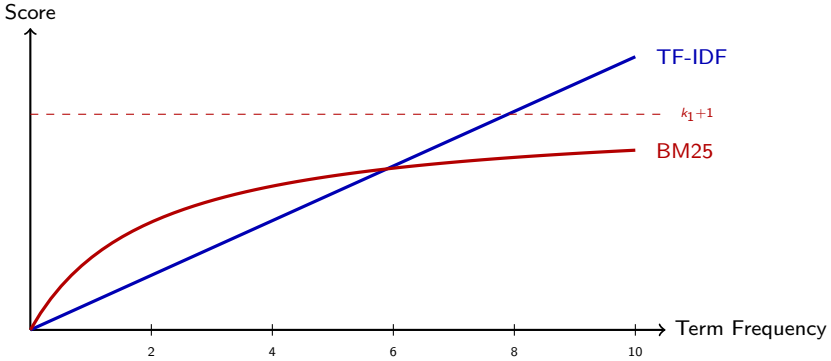
Parameters:

- k_1: saturation parameter (typically 1.2–2.0)
- b: length normalization (typically 0.75)
- avgdl: average document length in the corpus

In Practice

BM25 is the default ranking function in Elasticsearch, Solr, and most modern search engines. If you're doing lexical search, you're almost certainly using BM25.

BM25 vs. TF-IDF: The Saturation Curve



- **TF-IDF**: score grows **linearly**—the 100th occurrence of a term is weighted 10× more than the 10th
- **BM25**: score **saturates**—after ∼5–10 occurrences, additional matches barely increase the score

The "API" of Lexical Search

We can think of lexical search at three levels of abstraction:

1. **Single field:**

```
def lexical_search(query: str, field: str, doc_type: DocType)
    -> list[tuple[DocId, float]]: ...
```

2. **All text fields** (combine via "boost" weights—linear combination):

```
def lexical_search(query: str, doc_type: DocType)
    -> list[tuple[DocId, float]]: ...
```

3. **All document types** (hard—BM25 scores aren't comparable across corpora; use z-score or min-max normalization):

```
def lexical_search(query: str) -> list[tuple[DocId, float]]: ...
```

Outline

1. What Is Search?
2. Lexical Search
3. Semantic Search
4. Hybrid Search
5. Agentic Search
6. Non-Query Signals
7. Discussion
8. Conclusion

What Is Semantic Search?

Semantic Search uses the **meaning** of queries to search against the **meaning** of documents.

Example

Query: "I'm growing a forest and only want tall trees, find me those"

doc1: {name: "Opera Singers throughout History", content: "many opera singers existed..."}
doc2: {name: "Oaks and Pines", content: "these fine plants are very large"}

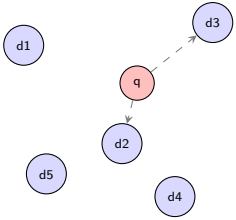
Intuition: doc2 should match and rank higher—even though "tall trees" appears in neither document. Lexical search would struggle here; semantic search understands meaning.

The Key Idea: Embedding Space

Imagine a very high-dimensional vector space of "ideas."

1. Turn documents into **vectors** in this space using an **encoder**
2. At query time, encode the query into the same space
3. Find documents whose vectors are **nearest neighbors** to the query vector

In practice, the encoder is a **transformer language model** that accepts strings as input and produces normalized vectors as output. Typical dimensionality: $\sim$1024.



embedding space

Efficiency

Document vectors can be **pre-computed** offline. At query time, only one embedding call is needed—for the query string.

Indexing for Semantic Search

Storage: D documents $\times$ DIM dimensions $\times$ 4 bytes (f32) $=$ total memory.

For 1M documents at 1024 dims: $1{,}000{,}000 \times 1024 \times 4 =$ **4 GB**.

The core operation: compute **nearest document neighbors** for a query vector.

Distance metrics: Euclidean, cosine similarity, inner product.

Naïve approach: Compare the query to *every* document vector, then sort.
$\Rightarrow$ This is expensive at scale.

Solution: Approximate Nearest Neighbors (ANN)

Build an index structure on top of the raw vectors that quickly identifies "potentially good" neighbors. Trades a small amount of recall for large speedups.

Vector ANN Indices

An **ANN index** creates structure on top of raw vectors for fast approximate neighbor lookup.

Common index structures:

- **HNSW** (Hierarchical Navigable Small World)—graph-based, very popular
- **IVF** (Inverted File Index)—partition-based
- **Product Quantization**—compression-based

Trade-off: There is a systematic chance of missing the true best neighbors (controlled by a recall parameter), but this allows massive query acceleration.

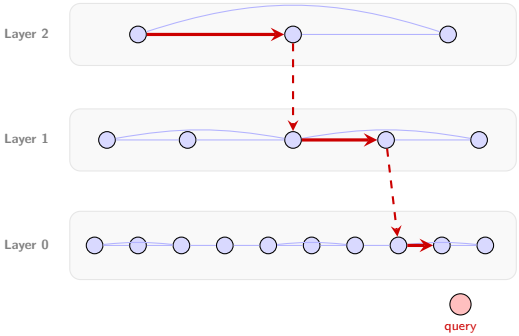
Popular vector databases:

- LanceDB
- Pinecone
- Weaviate
- Qdrant
- Milvus
- pgvector (Postgres)

Key Parameter

The **recall** parameter: how thoroughly to search the index. Higher recall = more accurate but slower.

HNSW: Intuition



Key idea: A multi-layer graph where each layer is a "navigable small world."

1. **Top layers** are sparse with long-range edges for coarse navigation
2. **Bottom layers** are dense with short-range edges for fine search
3. Search enters at the top and greedily descends, narrowing in on the nearest neighbor

Like a skip list, but for high-dimensional vector space.

Turning Documents into Strings for Embedding

A key detail: how do we turn rich, structured objects into **single strings** for the embedding model?

This is document-specific. Different document types need different strategies:

- **Simple documents:** Concatenate key text fields (name + description)
- **Tabular data:** Serialize column names and sample values
- **Complex documents (e.g. notebooks):** May need special treatment

Advanced: HyDE (Hypothetical Document Embedding)

Instead of embedding raw content, feed it to an LLM and ask: "What questions could this document answer?" Then embed *those questions* instead of the raw content.

This aligns the document's embedding with the kinds of queries users will actually issue.

HyDE Prompt Example

```
You are a data science manager with a deep understanding of
Python and SQL. Your task: given a project's metadata and
contents, generate questions that the project COULD ANSWER.

Respond as a JSON array of strings.

Example questions for a "Customer Growth" project:
1. How many customers do we have this year?
2. What was our customer growth over the last year?
3. How many new customers did we add per month?
4. What is the average customer acquisition cost per campaign?
5. What are the key factors influencing customer retention?

Here is the project status: {STATUS}
Here are the project categories: {CATEGORIES}
Here are the project contents: {CELL_CONTENTS}

Generate five questions. Each question: concise, max two sentences.
```

The generated questions are then embedded by the embedding model—*not* the raw cell contents.

Outline

1. What Is Search?
2. Lexical Search
3. Semantic Search
4. Hybrid Search
5. Agentic Search
6. Non-Query Signals
7. Discussion
8. Conclusion

What Is Hybrid Search?

Hybrid Search combines **lexical** and **semantic** search results, using the strengths of both.

The Setup

Run both search strategies independently:

```
lexical_results:  list[tuple[DocId, float]]
semantic_results:  list[tuple[DocId, float]]
```

The Problem

Scores are **incomparable**—BM25 scores have no bounded range; cosine similarities are in $[0, 1]$.

We need a **score-agnostic** merge strategy.

Option 1: We Don't Care About Order

Simplest approach: Just merge the sets of document IDs.

```
def merge_results(
    v1: list[tuple[DocId, float]],
    v2: list[tuple[DocId, float]],
) -> set[DocId]:
    return {doc_id for doc_id, _ in v1} | {doc_id for doc_id, _ in v2}
```

Result: at most `limit_lexical` + `limit_semantic` documents (with deduplication).

When Is This OK?

In the agentic world, it's actually unclear whether presentation order of search results matters at all—the agent processes all of them regardless.

Option 2: Reciprocal Rank Fusion (RRF)

If order matters, a simple and effective merge strategy is **Reciprocal Rank Fusion**.

Key idea: Documents near the top of *both* result lists are likely the best overall.

For each document d appearing at rank r in a result list:

$$\mathrm{RRF}(d) = \sum_{\text{list } \ell} \frac{1}{K + r_\ell(d)}$$

where $K = 60$ is a constant that works well empirically.

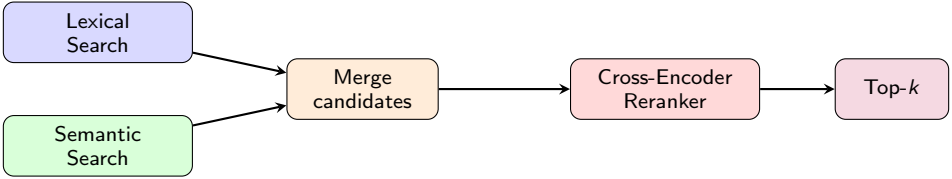
Example

DocId	Lexical Rank	Semantic Rank	RRF Score
10	1	2	$\frac{1}{61} + \frac{1}{62} = 0.0326$
19	4	1	$\frac{1}{64} + \frac{1}{61} = 0.0320$
17	5	4	$\frac{1}{65} + \frac{1}{64} = 0.0310$

Advantage: Not sensitive to the score distribution of either search method.

Option 3: Cross-Encoder Reranking

For the highest quality ordering, use a **cross-encoder reranker** model.



How it works:

- The reranker is a transformer that is **query-aware**—it performs full attention over query *and* document jointly
- API: `rerank(query: str, documents: list[str]) -> list[int]`
- Use lexical + semantic search for first-stage candidate retrieval, then pass all candidates through the reranker

Trade-off

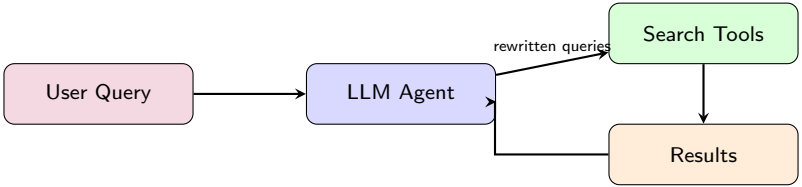
Computationally expensive—can't run over the full corpus. Use it as a second stage on a small candidate set.

Outline

1. What Is Search?
2. Lexical Search
3. Semantic Search
4. Hybrid Search
5. Agentic Search
6. Non-Query Signals
7. Discussion
8. Conclusion

What Is Agentic Search?

Equip an LLM agent with search tools and let it **call them in a loop** with different arguments.



Search in a Loop

This is fundamentally different from classical search.

- The agent takes an initial user query as input
- It can issue searches—hybrid, semantic, or lexical—**in parallel**
- Critically, the tool calls take **agent-provided search queries** as inputs

Key Insight

The agent doesn't pass the original user query into the search tools—it gets to **rewrite its own queries** based on:

- The original user query
- Whatever it has learned during the course of its execution
- Results from previous search iterations

This is how modern search agents work: the actual user query is not used directly. The agent decides what to search for.

Agentic Search: Worked Example

User: "What were our top-performing product lines last quarter?"

Turn 1

Searches: `semantic("product line revenue Q3 2025") | lexical("quarterly performance")`
Finds: Q3 revenue dashboard, product catalog

↓

Turn 2

Searches: `semantic("Q3 2025 revenue breakdown by product category")`
Finds: Detailed breakdown report with per-line revenue

↓

Turn 3

Agent synthesizes all retrieved documents and responds to the user.

Key Observation

None of the search queries match the user's original phrasing. The agent **decomposed the intent** and issued targeted, domain-specific queries—refining based on what it found.

Outline

1. What Is Search?
2. Lexical Search
3. Semantic Search
4. Hybrid Search
5. Agentic Search
6. Non-Query Signals
7. Discussion
8. Conclusion

Beyond the Query

Even the best query isn't the whole picture.

The documents in a user's workspace carry **intrinsic signals**—metadata, structure, and usage patterns—that are valuable for ranking *independent* of any query.

Implicit signals:

1. **Recency:** Entities opened, executed, or modified more recently are more likely to be relevant
2. **Authority:** Entities with endorsed statuses, published versions, or stars are likely more relevant
3. **PageRank:** Entity importance as a graph eigenvalue property

More implicit signals:

4. **Personalization:** If a user consistently uses an entity, it may be conditionally more relevant to *them*
5. **Search history:** Repeated searches for the same things indicate importance
6. **Relational context:** Leverage entity relationships in a knowledge graph—"people also search for"

Outline

1. What Is Search?
2. Lexical Search
3. Semantic Search
4. Hybrid Search
5. Agentic Search
6. Non-Query Signals
7. Discussion
8. Conclusion

Discussion Questions

1. **From "find information" to "complete a task":** Traditional search optimizes for returning relevant documents quickly. How should we rethink success metrics and product design when the goal shifts to task completion? What gets lost and gained?
2. **Value vs. compounding errors in agentic search:** Agentic search can chain together multiple queries and synthesize across sources. When does this create value versus when does it introduce compounding errors? How do we design for transparency and control in multi-step flows?
3. **Shallow vs. deep:** Agents could exhaustively research a topic, but users often just need a quick answer. How should we calibrate depth? Should it be user-controlled, context-dependent, or learned? What are the risks of getting this wrong?

Outline

1. What Is Search?
2. Lexical Search
3. Semantic Search
4. Hybrid Search
5. Agentic Search
6. Non-Query Signals
7. Discussion
8. Conclusion

Key Takeaways

1. **Search = documents + queries + ranking**
   - The core abstractions are simple; the details are where quality lives
2. **Lexical search matches on text; semantic search matches on meaning**
   - TF-IDF $\rightarrow$ BM25 for lexical; embedding models + ANN indices for semantic
3. **Hybrid search combines both for better coverage**
   - Merge strategies: set union, RRF, cross-encoder reranking
4. **Agentic search lets LLMs rewrite and iterate on queries**
   - The agent decides what to search for—not the user directly
5. **Non-query signals matter**
   - Recency, authority, personalization, and graph structure all improve relevance

Further Reading

Foundational:

- Robertson & Zaragoza (2009) — "The Probabilistic Relevance Framework: BM25 and Beyond"
- Karpukhin et al. (2020) — "Dense Passage Retrieval for Open-Domain Question Answering"
- Malkov & Yashunin (2020) — "Efficient and Robust Approximate Nearest Neighbor using Hierarchical Navigable Small World Graphs" (HNSW)

Hybrid & Reranking:

- Cormack et al. (2009) — "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods"
- Nogueira & Cho (2019) — "Passage Re-ranking with BERT"
- Gao et al. (2022) — "Precise Zero-Shot Dense Retrieval without Relevance Labels" (HyDE)

Thank you!